

DevDash'26

UCL One

Team Name: Sapiens

Team Members: Vishal, Shamal, Piushan, Isuru

Submission Date: 19/09/2026

GitHub Repository: github.com/DevDash-26/sapiens

1. Executive Summary

UCL One is a unified campus app that gives University of Lancashire students one reliable, role-aware channel for everything from announcements and events to room bookings and lost & found — replacing the fragmented mix of WhatsApp groups, notice boards, and word-of-mouth they rely on today. It solves the core problem of scattered, untrustworthy information by centralizing campus communication into a single, permission-controlled app that reduces the administrative burden on staff while making student life more visible and accessible. We're proud of the AI Assistant, which lets students get instant, conversational answers to campus questions instead of manually digging through menus — turning a static information app into something that genuinely feels helpful on first use.

2. Problem Understanding

2.1 Business Background

At University of Lancashire, there is no single source of truth for campus information — students piece together what they need from WhatsApp groups, lecturers, notice boards, society channels, and word of mouth, with no way to know which source is current or authoritative. This fragmentation means important updates like exam schedule changes or emergency alerts can easily be missed simply because a student wasn't in the right group chat, while routine questions about services, bookings, or society activities repeatedly end up as manual enquiries to administrative staff. The result is a system that wastes time on both sides: students spend unnecessary effort hunting for basic information, and staff are pulled into a fallback help desk for things that could easily be self-served if the information lived in one trusted, well-organized place.

3. Team & Roles

Name	Role	Key Responsibilities
Vishal	Front-end Developer	- Converted the UIs into a working application - Connected the Front-end to the Back-end
Piushan	UI/UX Designer	- Designed the UI screens and finalized the designs - Created the App logo
Shamal	Back-end Developer	Created the APIs

Isuru	QA Engineer	• E2E tested the app
-------	-------------	----------------------

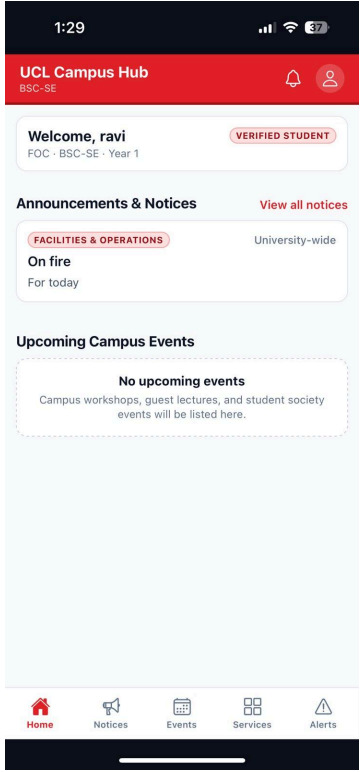
4. Solution Overview

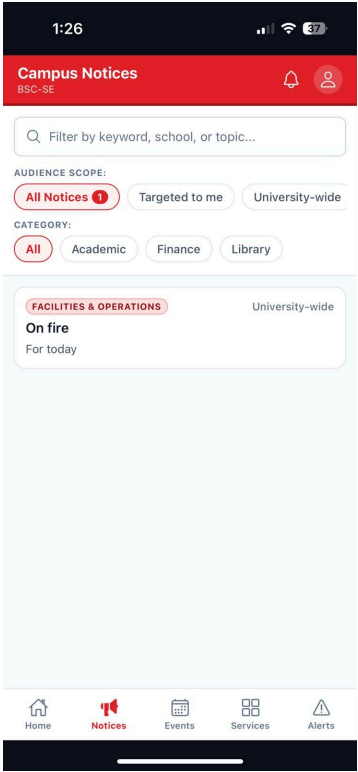
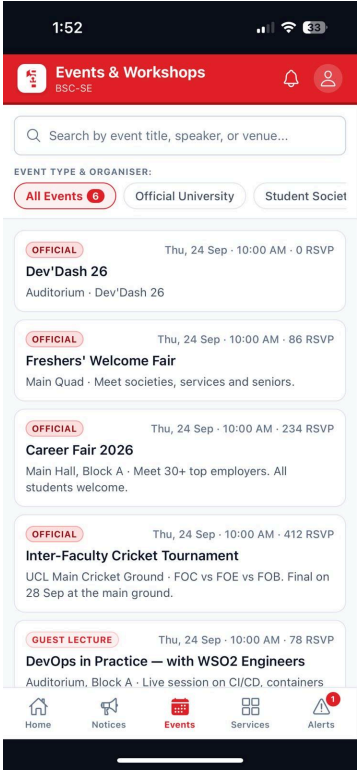
4.1 High-Level Concept

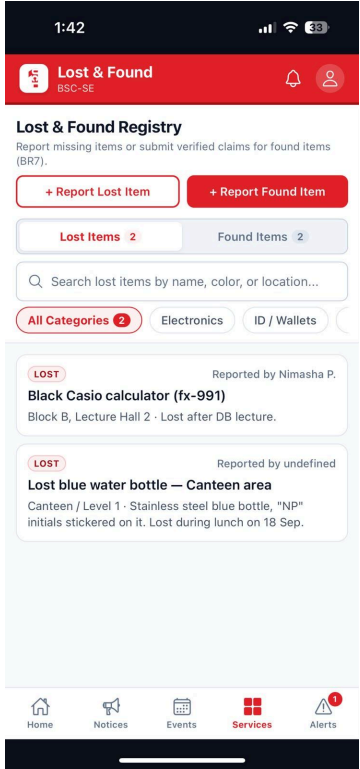
UCL One is a responsive web application designed to serve as the University of Lancashire's single digital front door for campus life. Its primary users are students — across all faculties, year groups, and programmers — who need fast, trustworthy access to announcements, events, societies, and campus services, alongside university staff (academic, society, finance, and administrative roles) who need a straightforward way to publish and manage the information they're responsible for. Rather than replacing any one existing tool, UCL One's core value proposition is consolidation: it brings previously scattered channels into one role-aware, easy-to-navigate platform, so students always know where to look and staff spend less time fielding questions that the system can now answer on its own.

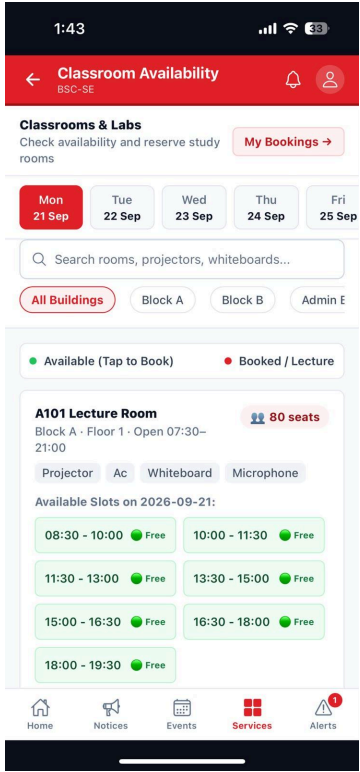
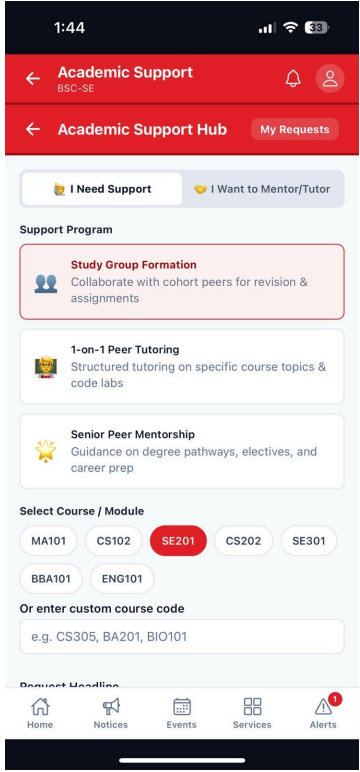
5. Requirement Coverage

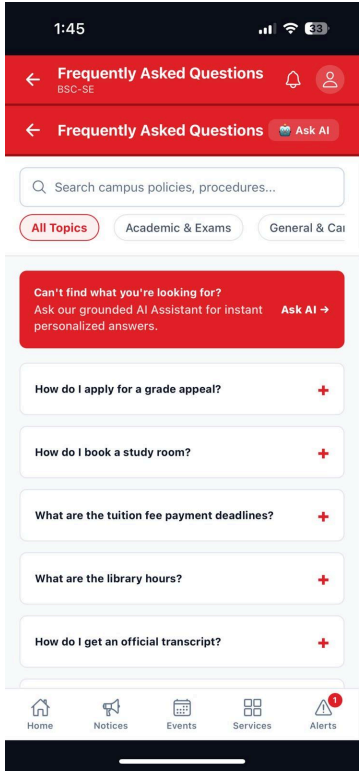
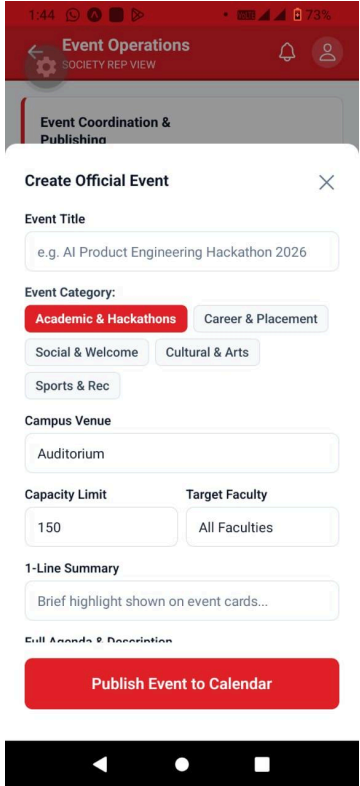
5.1 Business Requirements

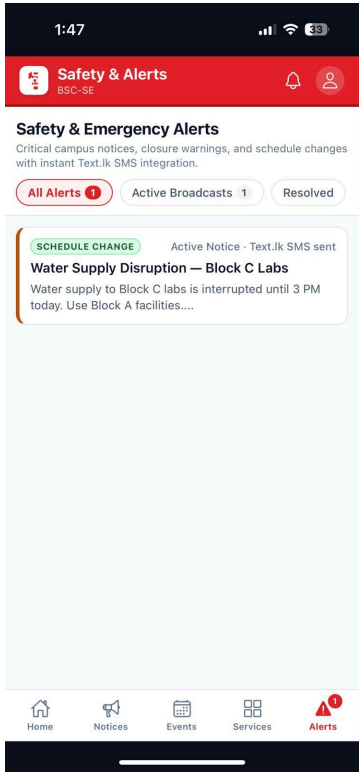
ID	Requirement	Marks	Status	Notes / Evidence
BR1	Unified Access	5	Done	

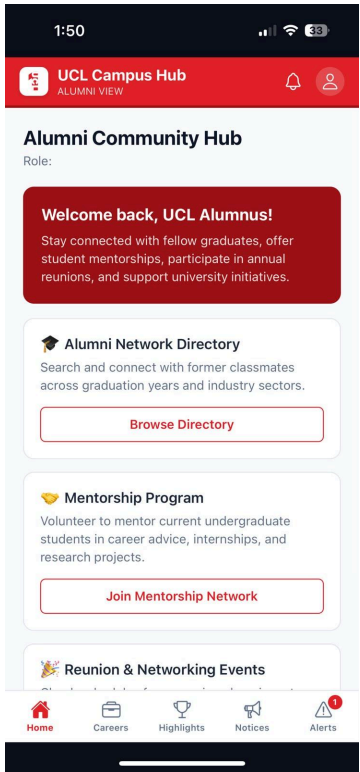
BR2	Targeted Announcements	3	Done	
BR3	Event Visibility	3	Done	

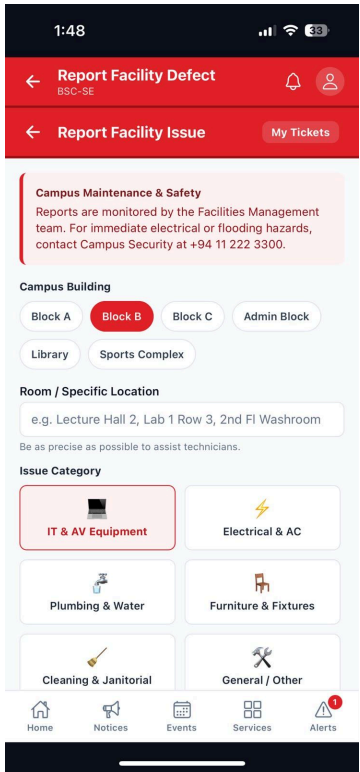
BR4	Event Interest	1	Done	
BR5	Society Visibility	3	☐ Done ☐ Partial ☐ N/A	
BR6	Society Sign-up	2	☐ Done ☐ Partial ☐ N/A	
BR7	Lost & Found	3	Done	

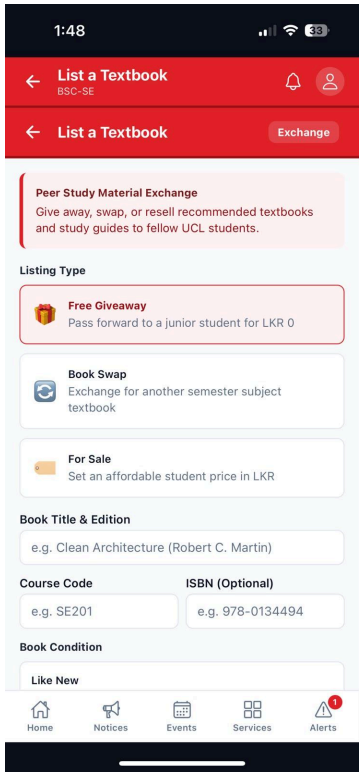
BR8	Classroom Booking	5	Done	
BR9	Academic Support	3	Done	

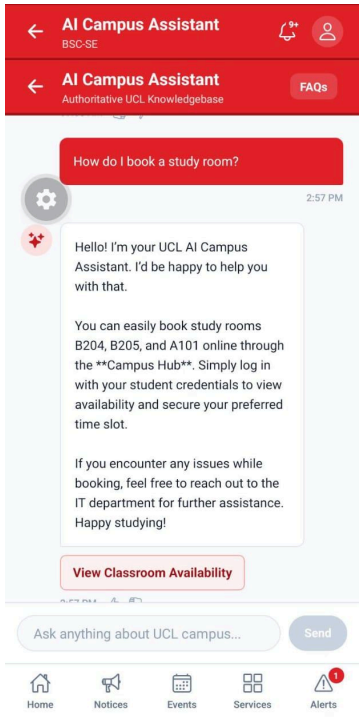
BR10	FAQ Access	2	Done	
BR11	Content Maintenance	4	Done	

BR12	Access Levels	6	☐ Done ☐ Partial ☐ N/A	
BR13	Academic Calendar	3	☐ Done ☐ Partial ☐ N/A	
BR14	Student Onboarding	2	☐ Done ☐ Partial ☐ N/A	
BR15	Emergency Communication	3	Done	
BR16	Schedule Changes	1	☐ Done ☐ Partial ☐ N/A	
BR17	Feedback Loop	1	☐ Done ☐ Partial ☐ N/A	
BR18	Volunteering Opportunities	1	☐ Done ☐ Partial ☐ N/A	

BR19	Alumni Engagement	1	Done	
BR20	Job & Internship Visibility	3	☐ Done ☐ Partial ☐ N/A	

BR21	Facility Issue Reporting	2	Done	
BR22	Staff Directory	2	Partial	
BR23	Financial Support Info	3	Partial	
BR24	Sports & Recreation	2	☐ Done ☐ Partial ☐ N/A	
BR25	Dining Information	1	☐ Done ☐ Partial ☐ N/A	
BR26	Printing Services	1	☐ Done ☐ Partial ☐ N/A	

BR27	Textbook Exchange	1	Done	
BR28	Guest Lectures	1	☐ Done ☐ ☐ Partial ☐ ☐ N/A	
BR29	Wellbeing Support	3	☐ Done ☐ ☐ Partial ☐ ☐ N/A	
BR30	IT Support Info	2	☐ Done ☐ ☐ Partial ☐ ☐ N/A	
BR31	Library Resources	2	☐ Done ☐ ☐ Partial ☐ ☐ N/A	
BR32	Student Life Highlights	1	☐ Done ☐ ☐ Partial ☐ ☐ N/A	

BR33	AI Assistant	9	Done	
------	--------------	---	------	---

5.2 Non-Functional Requirements

ID	Quality Attribute	Marks	How We Addressed It
NFR1	Usability	2	Recognizing that students do not constantly log into websites without access to laptops or desktops, we built a cross-platform mobile app with web access enabled, offering the convenience and feature set of both worlds.
NFR2	Performance & Scalability	3	Firestore database and buckets are hosted on Asian servers for minimal latency, supported by simple Firebase caching. The 3-tier architecture ensures smooth scalability by allowing individual components to expand independently, while React Native provides strong developer support and high availability across platforms.

NFR3	Reliability & Availability	1	Reliability is ensured through regular Firestore backups and automated failover capabilities. High availability is supported by Google Cloud's auto-scaling infrastructure, enabling 24x7 global accessibility with internet access. Emergency alert availability is further extended using Text.lk SMS integration, delivering critical notifications directly to mobile devices even when Wi-Fi is unavailable.
NFR4	Security & Privacy	2	• JWT-Based Access Control: Secure authentication using JSON Web Tokens ensures user identity is verified across requests. • Role-Based Access Control (RBAC): Strict permissions and role tiers ensure users only access and manage data relevant to their role. • Database Backups on Firebase: Automated database backups on Firebase protect against data loss and ensure system resilience. Rate Limiting: Request rate limiting prevents API abuse and mitigates denial-of-service attempts.
NFR5	Maintainability	4	Maintainability is achieved through separated codebases following clean architecture principles, supported by a CI/CD pipeline for automated Firebase deployment.
NFR6	Robustness	3	Robustness is achieved through comprehensive client and server-side input validation, strict TypeScript type enforcement across the monorepo, centralized error-handling middleware in Express REST APIs, and graceful fallback handling for third-party integrations and network interruptions.

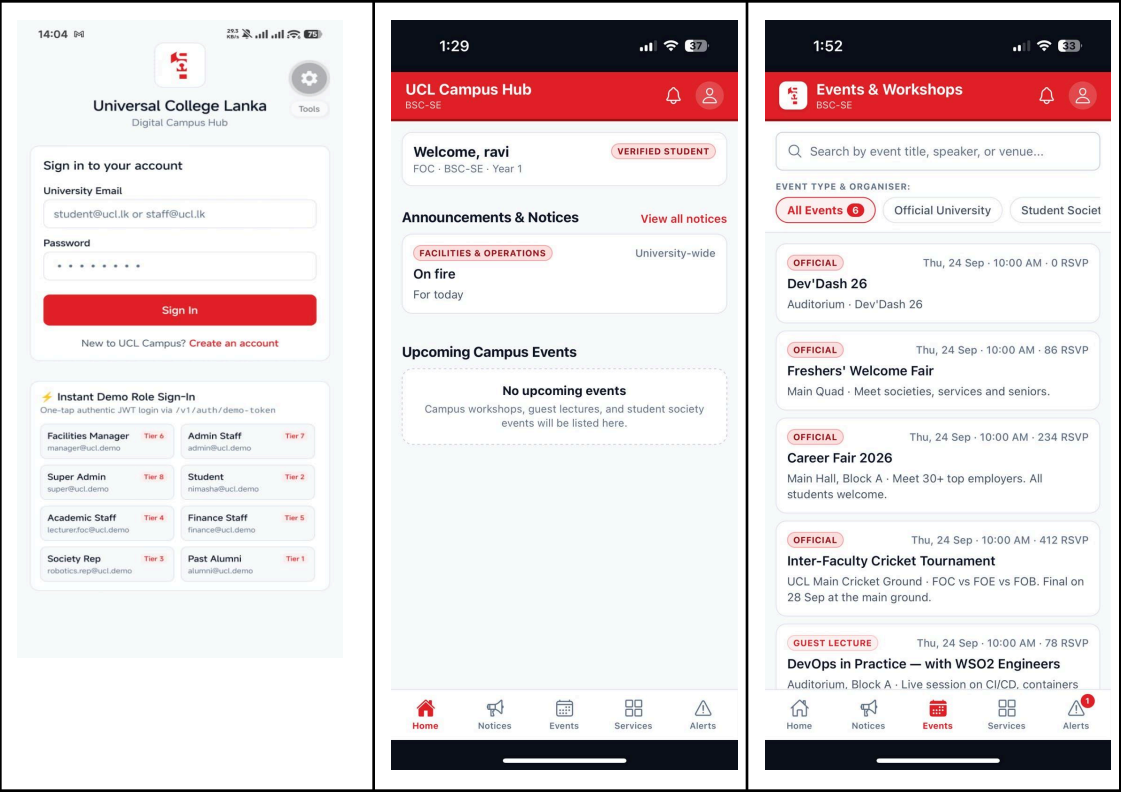
6. Design Process

6.1 User Research / Assumptions

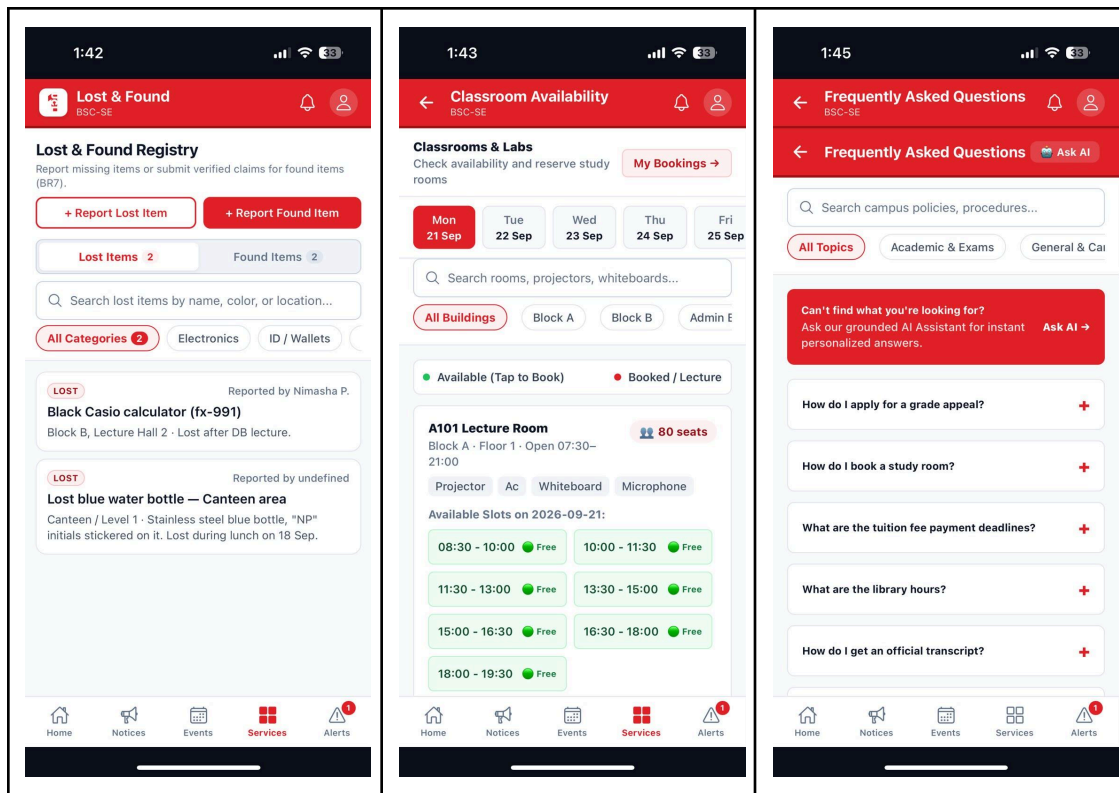
- **Student:** Assumed to need a fast, filtered feed relevant to their own faculty and year group, plus self-service actions like room booking, event RSVPs, and lost & found reporting — without needing to understand the system's underlying structure.
- **Society Rep / Club President:** Assumed to have limited technical confidence and time, so privileges are narrow — managing their own society profile, submitting events for review, and viewing registrations — rather than full publishing control.
- **Academic Staff:** Assumed to need speed and relevance to their own department, so they can publish faculty-specific notices like timetable revisions and industry talks without platform-wide visibility.
- **Finance Staff:** Assumed sensitive financial information should stay siloed from general content, so this role publishes financial notices and manages fee-related data within its own scope.
- **Manager:** Assumed operational decisions like room-booking approvals need a review step rather than direct student self-service, so this role acts as an approvals layer above content creation.
- **Admin Staff:** Assumed only a small, accountable group should control emergency communication and access provisioning, so this role posts campus-wide alerts, pins notices, and assigns user roles.
- **Super Admin:** Assumed full CRUD access across all collections should be reserved strictly for system-level maintenance, not everyday content operations.
- **Alumni:** Assumed to need a lighter-weight experience focused on directory access, mentorship, networking, and reunion events, reflecting UCL's goal of extending engagement beyond graduation.

6.2 Wireframes / Mockups

Sign Up Screen	Student Homescreen	Event Display Screen
----------------	--------------------	----------------------



Lost & Found Screen	Classroom Availability Screen	FAQ Screen
---------------------	-------------------------------	------------



6.3 Design System

- **Color palette:** Red, navy, and white drawn from the UCL identity — red for primary actions and urgent cues (Sign In button, active nav tabs, alert badges), navy/charcoal for text, and white/light-grey backgrounds for clarity.
- **Typography:** Clear hierarchy with bold headings for screen titles and lighter text for secondary details, allowing users to scan screens quickly.
- **Buttons & chips:** Pill-shaped buttons and filter chips used for building selectors, day tabs, and issue categories.
- **Cards:** Card-based containers group related content (room listings, issue categories, alumni programs).
- **Status indicators:** Color-coded dots (green = available, red = booked) instead of relying on text alone.
- **Navigation:** Role-adaptive bottom navigation bar that changes per user type (e.g., Home/Notices/Events for students vs. Home/Careers/Highlights for alumni).
- **Why it fits:** Consistent, color-coded, low-text-reliant design suits a university audience with varying digital literacy — from first-year students to occasional alumni users.

7. System Architecture

7.1 Technology Stack

- **Frontend:** Expo (SDK 57), React Native / React Native Web — deployed as a Web SPA on Netlify
- **Backend:** Node.js (TypeScript) on Firebase Cloud Functions (Express REST API)
- **Database:** Google Cloud Firestore (NoSQL)
- **Authentication:** Firebase Authentication + Custom Claims (RBAC)
- **Storage:** Google Cloud Storage
- **SMS Gateway (BR15):** Text.lk API
- **DevOps:** Monorepo (npm workspaces), TypeScript strict mode, Postman API testing

7.2 Architecture Diagram

UCL One follows a standard 3-tier architecture designed to separate concerns, ensure security, and maintain high responsiveness:

- **Presentation Tier (Client):** A responsive web client built with React Native and styled for mobile-first user experience. It handles user interactions, role-adaptive navigation, dynamic dashboard rendering, and client-side input validation.
- **Application / Server Tier (Backend):** A RESTful API services layer (Node.js/Express) responsible for business logic, JWT authentication, Role-Based Access Control (RBAC) enforcement, request rate-limiting, and routing queries to external integrations (e.g., Text.lk SMS gateway for emergency alerts and AI services for BR33).

Database Tier (Data Store): A managed cloud database hosted on Firebase Firestore for scalable, real-time data persistence, storing collection data for users, announcements, events, room bookings, lost & found items, and facility reports with automated backups.

8. Key Decisions & Trade-offs

- **Foundational App vs. Advanced UX Refinement:** Focused primarily on core features and building a solid foundational mobile application rather than exhaustive UX polishing, as tight hackathon time constraints limited advanced UX refinement and micro-interactions.
- **Simplified Development Infrastructure:** A dedicated branch-based development and staging infrastructure could not be fully established within the timeframe, highlighting an operational area reserved for future CI/CD pipeline enhancements.

MVP Scope vs. Feature Completeness: Prioritized delivering a fully functional Minimum Viable Product (MVP) foundation covering essential business requirements that the university can reliably build upon, rather than attempting to perfect every secondary peripheral feature.

9. Testing & Validation

9.1 Testing Approach

Our testing strategy focused on comprehensive end-to-end verification across the entire application flow. We systematically navigated from screen to screen, manually validating all interactive UI elements including buttons, navigation tabs, inputs, and action triggers. Every feature was tested to confirm that functional logic operates as expected, and data integration was thoroughly checked to ensure dynamic content, search filters, and API responses accurately render and persist across user interactions.

9.2 Test Log

Feature / BR Tested	Test Type	Expected Result	Actual Result
BR1: Unified Access	Functional / E2E	User authenticates via JWT and lands on role-specific home dashboard	Pass: Dashboard renders appropriate student navigation and announcements
BR2: Targeted Announcements	Integration	Filter notices by scope (Targeted to me vs. University-wide)	Pass: Notices filtered correctly according to student faculty/year scope
BR8: Classroom Booking	Functional / E2E	Select date, building, and free time slot to request room reservation	Pass: Room slot updates status and booking request submits for review
BR15: Emergency Communication	System / Integration	Broadcast urgent alert to Safety & Alerts tab with instant SMS trigger	Pass: Critical alert appears in feed and SMS gateway delivers alert message

9.3 Known Issues / Limitations

While core functionalities are operational, several areas require further refinement for a seamless production experience:

- **App Flow Improvements:** Smooth navigation transitions and state preservation when switching between tabs (e.g., maintaining filter states in Notices and Events) need further optimization to avoid unintended resets.
- **UX Enhancements:** Additional visual feedback mechanisms (such as skeleton loaders during API fetch delays), improved form validation messaging, and enhanced accessibility support for screen readers are scheduled for future iterations.

10. Security & Privacy Considerations

- **JWT-Based Access Control:** Secure authentication using JSON Web Tokens ensures user identity is verified across requests.
- **Role-Based Access Control (RBAC):** Strict permissions and role tiers ensure users only access and manage data relevant to their role.
- **Database Backups on Firebase:** Automated database backups on Firebase protect against data loss and ensure system resilience.
- **Rate Limiting:** Request rate limiting prevents API abuse and mitigates denial-of-service attempts.
- **N-Tier Architecture:** Separation of concerns across multi-tier architecture isolates core application layers and enhances overall security.

11. Challenges & Learnings

The hardest decision was scoping with 33 business requirements and limited hackathon time, deciding what to build deeply versus what to leave lightweight (e.g., prioritizing the AI Assistant and 8-tier role system over lower-mark features like Textbook Exchange) required constant trade-off discussions. Technically, implementing role-based access control correctly across 8 distinct user tiers was the most complex piece, since every feature needed permission checks enforced consistently rather than just hidden in the UI. Next time, we'd start architecture and role-permission design earlier, before feature development, to avoid retrofitting access control into screens that were already built.

12. Future Roadmap

Beyond the hackathon, we'd prioritize integrating UCL One with existing university systems (student records, timetabling) to auto-populate targeted announcements and reduce manual data entry by staff. We'd also expand the AI Assistant with richer, context-aware responses trained on UCL's actual FAQ and policy documents, add push notifications for real-time alerts, and build an analytics dashboard so admin staff can track engagement and identify gaps in campus communication over time.